

Client Telemetry Project - Test Cases

Standalone test case extract from the System Test Plan. The original section number has been removed and the layout has been cleaned for readability.

27	18	6	3	24
Total test cases	High priority	Medium priority	Low priority	Completed / passed

TC 1 SESSION_DATA_LOAD		Priority: High
Title	Session Data Loads Correctly	
Description	Verify that session data is correctly loaded and displayed in the Session Detail page.	
Pre-condition	Backend server is running and session data exists in S3.	
Action(s) / Steps	1. Open dashboard 2. Navigate to Sessions page 3. Click on a session	
Post-conditions	sessionId, timestamp, event count, and score are displayed correctly with no missing values	
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)	
Test Status	Pass - Dashboard loads successfully and session data is retrieved from the backend and displayed correctly. All required fields (session ID, score, severity, status, events, timestamp) are populated with no missing values.	

TC 2 AGG_METRICS_ACCURACY		Priority: High
Title	Aggregated Metrics Accuracy	
Description	Verify that aggregated metrics displayed in the UI match backend API values.	
Pre-condition	Session data with computed metrics exists.	
Action(s) / Steps	1. Open Session Detail page 2. Open browser dev tools -> Network tab 3. Compare API response with UI values	
Post-conditions	event_count, total_dwell_ms, and session_duration_ms match exactly between API and UI	
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)	
Test Status	Pass - Aggregated metrics displayed in the UI match backend API values. Verified using browser developer tools (Network tab), where the /api/sessions request returned a successful response (status 200), and event count and score values were consistent with those displayed in the UI.	

TC 3 | BEHAVIOR_METRICS_DISPLAY

Priority: Medium

Title	Behavior Metrics Display
Description	Verify that behavioral metrics are displayed correctly in the dashboard.
Pre-condition	Feature data exists for selected session.
Action(s) / Steps	1. Open Session Detail page 2. Locate Behavior Metrics section
Post-conditions	page_view_count, unique_route_count, rage_click_count, and flow_success_count are displayed correctly
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Behavior metrics are displayed correctly in the session detail view. Verified presence of metrics such as page views, unique routes, field changes, and session duration within the Behavior Metrics section.

TC 4 | PREDICTION_MAPPING

Priority: High

Title	Prediction Mapping Correctness
Description	Verify correct pairing between sessionId, feature data, and prediction results.
Pre-condition	Predictions and feature data exist for sessions.
Action(s) / Steps	1. Open Session Detail page 2. Check sessionId 3. Compare with prediction and feature data
Post-conditions	No mismatches between sessionId, feature data, and prediction results
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Session IDs are correctly mapped to their corresponding feature data and prediction results. Verified by selecting sessions and confirming consistency of session ID, metrics, and severity/score values between overview and detail views.

TC 5 | DATA_CONSISTENCY

Priority: High

Title	Data Consistency Across Dashboard Pages
Description	Verify that the same session displays identical data across all dashboard views.
Pre-condition	Session appears in multiple dashboard views.
Action(s) / Steps	1. Open Sessions page 2. Open Alerts page 3. Open Queue page 4. Open Session Detail page
Post-conditions	Same values (score, severity, timestamp, event count) appear across all pages
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Session data (score, severity, timestamp, and event count) is consistent across dashboard views. Multiple sessions were verified to ensure values remain accurate and consistent between the Sessions page and Session Detail view, with no discrepancies observed.

TC 6 | MISSING_DATA_HANDLING

Priority: Medium

Title	Missing Data Handling
Description	Verify that missing or incomplete data is handled properly in the UI.
Pre-condition	Session contains missing fields (e.g., scenario).
Action(s) / Steps	1. Load session with missing data 2. Observe UI behavior
Post-conditions	UI does not crash and displays fallback values (e.g., "--" or 0)
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - The UI handles missing or incomplete data correctly. Missing fields such as scenario and certain metrics are displayed using fallback values (e.g., "--", "-", or 0), and the dashboard remains stable without any crashes.

TC 7 | AUTO_REFRESH

Priority: Medium

Title	Auto-Refresh and Latest Data Loading
Description	Verify that dashboard loads latest data after new data is generated.
Pre-condition	New data is uploaded to S3.
Action(s) / Steps	1. Generate new telemetry data 2. Refresh dashboard
Post-conditions	Latest run data is displayed and no stale data appears
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: - (-)
Test Status	Not completed / not recorded

TC 8 | END_TO_END_PIPELINE

Priority: High

Title	End-to-End Pipeline Validation
Description	Verify that the full pipeline works from data generation to dashboard display.
Pre-condition	All phases are available and runnable.
Action(s) / Steps	1. Generate telemetry data (Phase 1) 2. Run preprocessing (Phase 2) 3. Load dashboard (Phase 3)
Post-conditions	Data flows correctly across all phases and appears in the dashboard
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: - (-)
Test Status	Not completed / not recorded

TC 9 | ALERT_QUEUE_LOGIC

Priority: High

Title	Alerts and Queue Functionality
Description	Verify that sessions are correctly classified and displayed in Alerts and Queue views, and that the Queue is sorted by severity and timestamp.
Pre-condition	Sessions with varying severity exist.
Action(s) / Steps	1. Open Alerts page 2. Open Queue page 3. Observe ordering of sessions in Queue
Post-conditions	High severity sessions appear in Alerts, and Queue is sorted (High -> Medium -> Low), and within each severity level, sorted by timestamp (most recent first)
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Alerts and Queue functionality works correctly. Medium and high severity sessions are displayed in the Alerts view, and the Queue is properly sorted by severity (High -> Medium -> Low), with correct prioritization observed.

TC 10 | EVENT_TIMELINE

Priority: Low

Title	Event Timeline Display
Description	Verify that raw events are correctly displayed in the event timeline.
Pre-condition	Raw event data exists and timeline is implemented.
Action(s) / Steps	1. Open Session Detail page 2. Scroll to Event Timeline
Post-conditions	Events are grouped by sessionId and ordered by timestamp
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: - (-)
Test Status	Not completed / not recorded

TC 11 | ADMIN_PAGE_DISPLAY

Priority: Medium

Title	Admin Page Displays Session Data
Description	Verify that selecting a session in the Admin page displays correct metadata.
Pre-condition	Dashboard is running and sessions exist.
Action(s) / Steps	1. Open Admin tab 2. Select a session from dropdown
Post-conditions	Session ID, severity, score, and timestamp display correctly
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Admin page displays selected session data correctly. Verified that session ID, severity, score, and timestamp are shown after selecting a session from the Admin page.

TC 12 | ADMIN_OVERRIDE_SEVERITY

Priority: High

Title	Override Severity Works Correctly
Description	Verify that an admin can override the model's severity value.
Pre-condition	Session is selected in Admin page.
Action(s) / Steps	1. Select a session 2. Change override severity (e.g., LOW -> HIGH) 3. Click "Save override"
Post-conditions	New severity is saved and reflected correctly
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Override severity functionality works correctly. Severity value can be updated through the Admin interface and is saved successfully without errors.

TC 13 | ADMIN_COMMENT

Priority: Low

Title	Admin Comment Input Works
Description	Verify that admin can add comments when overriding severity.
Pre-condition	Session selected.
Action(s) / Steps	1. Enter text in comment box 2. Click "Save override"
Post-conditions	Comment is saved without errors
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Admin comment input works correctly. Text can be entered and saved through the Admin interface without errors.

TC 14 | ADMIN_CLEAR_OVERRIDE

Priority: Low

Title	Clear Override Functionality
Description	Verify that clearing override resets values.
Pre-condition	-
Action(s) / Steps	1. Apply override 2. Click "Clear override"
Post-conditions	Override is removed and original values restored
Created / Tested	Created by: Geri Koka (04/19/2026) Tested by: Sabrina Akter (04/28/2026)
Test Status	Pass - Clear override functionality works correctly. Applied overrides can be successfully removed, and the session reverts to its original severity value.

TC 15 | P2-01

Priority: High

Title	Aggregated Metrics Accuracy
Description	[Component: preprocess.py] Verify that the aggregation logic correctly computes metrics from raw telemetry events.
Pre-condition	Raw telemetry data is available.
Action(s) / Steps	1. Run preprocess.py 2. Check event_count and session_duration_ms against raw data.
Post-conditions	Metrics match exactly; no NaN values.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. The aggregated metrics calculated is consistent and correct.

TC 16 | P2-02

Priority: High

Title	Severity Logic Correctness
Description	[Component: utils.py] Validate the mapping of frustration scores (0-1) to severity levels.
Pre-condition	utils.py is accessible for unit testing.
Action(s) / Steps	1. Test values <0.3 2. Test values 0.3 to 0.7 3. Test values >0.7 4. Test boundary values (0.299, 0.700)
Post-conditions	Severity properly maps to Normal, Medium, and High respectively.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. The severity scoring logic works correctly and consistently. Both utils and inference script are consistent with scoring logic.

TC 17 | P2-03

Priority: High

Title	Inference Workflow (Local)
Description	[Component: client.py] Verify local inference logic and the model syncing fallback mechanism.
Pre-condition	S3 bucket contains model artifacts.
Action(s) / Steps	1. Run client.py for inference 2. Simulate failed S3 download to test fallback.
Post-conditions	Inference succeeds, falling back to older artifacts if sync fails.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. The structure is verified with sync brain syncing the models appropriately.

TC 18 | P2-04

Priority: Medium

Title	Inference Workflow (SageMaker)
Description	[Component: serve.py] Verify the SageMaker real-time inference endpoint handles HTTP requests.
Pre-condition	Endpoint deployed and running.
Action(s) / Steps	1. Send JSON payload using test_endpoint.py.
Post-conditions	Returns 200 OK with scores and predictions JSON body.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. The inference workflow works as expected with endpoint handling HTTP requests.

TC 19 | P2-05

Priority: Medium

Title	Cloud Triggering (EventBridge)
Description	[Component: AWS Config] Validate automated pipeline triggering via EventBridge.
Pre-condition	EventBridge rule is configured.
Action(s) / Steps	1. Simulate Scheduled Event 2. Monitor invocation.
Post-conditions	Pipeline executes successfully, processing only new data since last run.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. Automated stimulation of scheduled event now works with eventbridge and processes data as needed.

TC 20 | P2-06

Priority: High

Title	Feature Mapping Consistency
Description	[Component: config/schema_default.json] Ensure mapping correctly standardizes multi-platform data.
Pre-condition	Raw telemetry from Web and Mobile.
Action(s) / Steps	1. Ingest Web and Mobile telemetry 2. Review output feature_vector.csv.
Post-conditions	Output perfectly matches the standardized schema.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. The schema structure matches the preprocessing script requirements.

TC 21 | P2-07

Priority: High

Title	Feedback Loop Optimization
Description	[Component: train.py] Verify semi-supervised feedback loop updates ensemble weights.
Pre-condition	Provide manually_labeled.csv.
Action(s) / Steps	1. Run train.py 2. Review model_metadata.joblib.
Post-conditions	Optimal weights are calculated and saved successfully.
Created / Tested	Created by: AI team (04/19/2026) Tested by: Manish Gurung (05/09/2026)
Test Status	PASS. Feedback loop is tested with dynamic threshold optimization and the validation logic has been verified.

TC 22 | TC-P1-00

Priority: High

Title	Telemetry Event Schema Validation
Description	Verify that all telemetry events follow the predefined schema format.
Pre-condition	Telemetry SDK is running and events are being generated.
Action(s) / Steps	1. Trigger user interactions (click, scroll, navigation) 2. Capture telemetry events 3. Inspect NDJSON logs
Post-conditions	All required fields exist (sessionId, eventType, timestamp, pageRoute, metadata). No missing or malformed fields.
Created / Tested	Created by: Data Team (04/19/2026) Tested by: Guanxin (6/01/2026)
Test Status	PASS. Telemetry event schema validation is tested, and all required fields, including sessionId, eventType, timestamp, pageRoute, and metadata, have been verified with no missing or malformed fields.

TC 23 | TC-P1-01

Priority: High

Title	Telemetry API Ingestion Success
Description	Verify that the backend API correctly receives and acknowledges valid JSON telemetry payloads.
Pre-condition	The Node.js ingestion server is running and accessible at the /api/telemetry endpoint.
Action(s) / Steps	1. Send a valid JSON payload (containing sessionId, eventType, etc.) to the API. 2. Observe the HTTP response.
Post-conditions	The server returns an HTTP 200 status code with a JSON response body: { "status": "success" }
Created / Tested	Created by: Data Team (04/27/2026) Tested by: Shengyang (6/01/2026)
Test Status	PASS. Telemetry API ingestion is tested with a valid JSON payload, and the backend response logic has been verified with HTTP 200 and { "status": "success" }.

TC 24 | TC-P1-02

Priority: High

Title	Local NDJSON Persistence
Description	Verify that received events are accurately appended to the local storage for downstream processing.
Pre-condition	TC-P1-01 has been executed successfully.
Action(s) / Steps	1. Access the logs/telemetry_logs.ndjson file on the server. 2. Verify the last appended line.
Post-conditions	The event is saved as a single-line JSON string appended with a serverReceivedAt timestamp.
Created / Tested	Created by: Data Team (04/27/2026) Tested by: Shengyang (6/01/2026)
Test Status	PASS. Local NDJSON persistence is tested, and the received telemetry event has been verified as a single-line JSON record appended with a serverReceivedAt timestamp.

TC 25 | TC-P1-03

Priority: High

Title	S3 File Rotation Mechanism
Description	Ensure that the system rotates logs correctly to prevent data loss during high-frequency uploads.
Pre-condition	The telemetry_logs.ndjson file exists and contains data.
Action(s) / Steps	1. Trigger the uploadLogsToS3 function manually or via interval. 2. Check the logs/ directory.
Post-conditions	The active log is atomically renamed to a timestamped temporary file, allowing the server to continue writing to a fresh log file.
Created / Tested	Created by: Data Team (04/27/2026) Tested by: Shengyang (6/01/2026)
Test Status	PASS. S3 file rotation is tested, and the log rotation mechanism has been verified to atomically rename the active log file while allowing continued logging to a fresh file.

TC 25 | TC-P1-04

Priority: High

Title	AWS S3 Tiered Synchronization
Description	Confirm that logs are successfully synced to the S3 bucket using the defined partitioned pathing.
Pre-condition	AWS credentials and bucket permissions are properly configured.
Action(s) / Steps	1. Initiate an automated or manual S3 sync. 2. Log into the AWS S3 console and navigate to the target bucket.
Post-conditions	The log file appears in the path raw/YYYY-MM-DD/telemetry_logs_TIMESTAMP.ndjson.
Created / Tested	Created by: Data Team (04/27/2026) Tested by: Shengyang (6/01/2026)
Test Status	PASS. AWS S3 tiered synchronization is tested, and the uploaded log file has been verified in the expected partitioned path format: raw/YYYY-MM-DD/telemetry_logs_TIMESTAMP.ndjson.

TC 26 TC-P1-05		Priority: High
Title	Graceful Shutdown Data Flush	
Description	Verify that all remaining local logs are flushed to S3 before the server process terminates.	
Pre-condition	The server is running and has un-synced logs in the local buffer.	
Action(s) / Steps	1. Send a SIGINT (Ctrl+C) or SIGTERM signal to the Node.js process.	
Post-conditions	The server captures the signal, executes uploadOnShutdown(), and confirms final sync completion before exiting.	
Created / Tested	Created by: Data Team (04/27/2026) Tested by: Shengyang (6/01/2026)	
Test Status	PASS. Graceful shutdown data flush is tested, and the shutdown handler has been verified to execute final log synchronization before the server process exits.	